CARBON DATA LOGGER

Personal Carbon Footprint Tracking System

Project Handbook and Session Record

CBSE Class XII Computer Science (083) · Python + MySQL

Contents
1. Project brief (the original requirements)
2. The finished project at a glance
3. How to run the project on your own device
4. Session record: what was asked and what was done
5. Key decisions, test results and known limits
6. Source code: database_setup.sql and carbon_logger.py

Repository: github.com/SG1222/Stuff · Updated 26 October 2026

1. Project brief

The project started from an agent brief (prompt.pdf) and the CBSE Class XII Computer Science syllabus for 2026–27 (CS_Board_Syllabus.pdf). The brief's main points were:

Objective

Build a terminal-based Python + MySQL program that lets one person record everyday activities that add to their carbon footprint (electricity, car/bus/train travel, LPG, flights). Python calculates the estimated CO2 with predefined emission factors and stores the records in MySQL. The user can search, edit, delete, summarise and export the data. The purpose is educational, so the emission factors must be labelled as illustrative.

Requirements

Area	What the brief asked for
Level	Within CBSE Class XI/XII concepts. No classes/OOP, decorators, generators, threading, APIs, web frameworks, ORMs, Pandas/NumPy, complex comprehensions or design patterns.
Size	About 500–650 meaningful lines across the Python and SQL files, without padding.
Interface	A neat terminal menu (13 options + Exit). No GUI, no login.
Database	Database carbon_logger with two related tables: categories (name, unit, emission factor) and emission_records (date, category, quantity, CO2, note), linked by a foreign key.
Calculation	CO2 = quantity × emission factor, in one function; factors read from MySQL; rounded to 2 decimals.
Features	Add (with confirmation), view (via JOIN), search (ID, category, date, date range), update (recalculate CO2), delete (with Y/N), overall/category/monthly summaries, highest category, rule-based suggestions, CSV export, text summary export, manage factors, help.
SQL to show	CREATE DATABASE/TABLE, PRIMARY/FOREIGN KEY, INSERT, SELECT, UPDATE, DELETE, WHERE, BETWEEN, ORDER BY, COUNT/SUM/AVG/MIN/MAX, GROUP BY, INNER JOIN.
Connectivity	mysql.connector with connect(), cursor(), execute(), commit(), fetchone(), fetchall(); parameterised %s queries.
Errors	try/except/finally for bad menu input, non-numeric/negative quantity, invalid dates, missing IDs, MySQL connection and file errors.
Files	carbon_logger.py (whole app) and database_setup.sql; the program generates carbon_records.csv and carbon_summary.txt.
Test	Could a Class XII student explain every part in a viva? If not, simplify.

2. The finished project at a glance

Repository files

File	Purpose
README.md (repo root)	Full explanation of the project, written in Simplified Technical English, including 22 'tricky parts' sections.
CarbonDataLogger/carbon_logger.py	The program: 45 functions, about 645 non-blank lines.
CarbonDataLogger/database_setup.sql	Creates the database, both tables, 6 default categories and 5 sample records.
CarbonDataLogger/VIVA_SHEET.md	62 likely viva questions with answers, plus a demo order.
CarbonDataLogger/Project_Report.docx	The CBSE project report (39 pages): certificate, design, code, 18 screenshots, 25 tests, bibliography.
CarbonDataLogger/Carbon_Data_Logger_Handbook.pdf	This handbook: brief, run guide, session record, tests and full code.
.gitignore	Keeps generated files (CSV, text summary, __pycache__) out of the repo.

Main menu

Option	Feature	Option	Feature
1	Add Carbon Record	8	Monthly Summary
2	View All Records	9	Highest Emission Category
3	Search Records (sub-menu)	10	Export Records to CSV
4	Update Record	11	Export Summary to Text File
5	Delete Record	12	Manage/View Emission Factors (sub-menu)
6	Carbon Footprint Summary	13	Help
7	Category-wise Summary	14	Undo Last Added Record (stack)
		0	Exit

Database design

Table	Columns	Keys
categories	category_id INT, category_name VARCHAR(30), unit VARCHAR(10), emission_factor FLOAT	PRIMARY KEY category_id (AUTO_INCREMENT); category_name UNIQUE
emission_records	record_id INT, record_date DATE, category_id INT, quantity FLOAT, co2_emission FLOAT, note VARCHAR(100)	PRIMARY KEY record_id (AUTO_INCREMENT); FOREIGN KEY category_id → categories

Default categories (illustrative factors, kg CO2 per unit): Electricity 0.82/kWh · Car Travel 0.17/km · Bus Travel 0.10/km · Train Travel 0.04/km · LPG 2.98/kg · Flight 0.15/km.

How the code is organised

- Constants at the top: MySQL settings (`DB_PASSWORD = "root"`), file names, default categories, suggestion texts, month names, and the shared RECORD_QUERY (SELECT … INNER JOIN).
- Setup: `connect_database()`, `create_tables()`, `insert_default_categories()`.
- Input helpers that repeat until the input is valid: `get_integer()`, `get_positive_number()`, `validate_date()`, `get_date()`.

- One function per feature (`add_record()`, `search_by_date_range()`, `monthly_summary()` …) and shared helpers such as `print_records()` and `get_category_totals()`.
- Undo uses a list as a stack with `push()`/`pop()`; `add_record()` pushes `cursor.lastrowid`.
- `main()` connects, then runs the menu loop through the dictionary `menu_actions`; database errors return to the menu; Ctrl+C and Exit close the connection in `finally`.

3. How to run the project on your own device

You need three things: Python 3, a MySQL server, and the Python package mysql-connector-python. The steps below are for Windows; macOS/Linux differences are at the end.

Step 1 – Install Python 3

1. Download Python 3 from python.org and run the installer.
2. On the first screen, tick **“Add python.exe to PATH”**, then click Install.
3. Check it: open Command Prompt and type `python --version`.

Step 2 – Install MySQL Server

1. Download **MySQL Installer (Community)** from dev.mysql.com and choose MySQL Server 8 (MySQL Workbench is optional).
2. During setup, set a **root password** and write it down. Using **root** means you don't need to change the code.
3. Leave “Start MySQL Server at System Startup” ticked so the server is running.

Step 3 – Get the project files

Either download the ZIP from GitHub (repository page → green **Code** button → **Download ZIP**, then extract it), or clone it:

```
git clone https://github.com/SG1222/Stuff.git
```

The program files are in the folder `CarbonDataLogger`.

Step 4 – Install the MySQL connector

```
pip install mysql-connector-python
```

If `pip` is not found, use `python -m pip install mysql-connector-python`.

Step 5 – Set your MySQL password

Open `carbon_logger.py` in IDLE or VS Code and find the line `DB_PASSWORD = "root"`. If your MySQL root password is not **root**, type your password between the quotes and save.

Step 6 (optional) – Load the sample data

This gives you 5 sample records to demonstrate straight away. Open Command Prompt in the CarbonDataLogger folder (in File Explorer, type **cmd** in the address bar and press Enter), then run:

```
mysql -u root -p < database_setup.sql
```

Enter your MySQL password when asked. If `mysql` is not recognised, open **MySQL Command Line Client** from the Start menu and type the SOURCE command with the full path, using forward slashes, for example:

```
SOURCE C:/Users/YourName/Downloads/Stuff/CarbonDataLogger/database_setup.sql;
```

Run the script only once. A second run stops with “Duplicate entry”. To start again, run `DROP DATABASE carbon_logger;` in the MySQL client and run the script again. If you skip this step, the program creates an empty database with the 6 categories by itself.

Use **MySQL 8**: its default character set (utf8mb4) lets notes contain ₹ and Hindi text. Older servers such as MySQL 5.7 default to latin1, which allows English letters only in notes.

Step 7 – Run the program

In the CarbonDataLogger folder:

```
python carbon_logger.py
```

Type a menu number and press Enter. Choose 0 to exit. Exported files (carbon_records.csv, carbon_summary.txt) are created in the folder you ran the program from.

macOS / Linux

- Install Python 3 and MySQL (macOS: from python.org and dev.mysql.com, or with Homebrew: `brew install python mysql`, then `brew services start mysql`).
- Use `python3` and `pip3` instead of `python` and `pip`.
- Load sample data with `mysql -u root -p < database_setup.sql` in Terminal.

Troubleshooting

What you see	Cause	Fix
ModuleNotFoundError: No module named 'mysql'	Connector not installed	pip install mysql-connector-python
Could not connect to MySQL … Access denied	Wrong password	Change DB_PASSWORD in carbon_logger.py
Could not connect to MySQL … Can't connect	MySQL server not running	Start it (Windows: Services → MySQL80 → Start)
'python' is not recognized	Python not on PATH	Use py carbon_logger.py, or reinstall with “Add to PATH” ticked
Duplicate entry … when running the SQL script	Data already loaded	DROP DATABASE carbon_logger; then run the script again
Database error … Incorrect string value (note with ₹ or Hindi)	Server older than MySQL 8 (latin1)	Use MySQL 8, or type the note in English letters only (README 10.22)
Invalid date! for a date that looks right	Date before 2000 or after today	Type a date from 2000 up to today, or press Enter for today
Error: could not write to carbon_records.csv	File open in Excel	Close the file, then export again
The < redirect fails in PowerShell	PowerShell syntax	Use Command Prompt (cmd), or the SOURCE command

4. Session record

Each of your messages is shown in a blue box, followed by a summary of what was done. This is a structured record of the conversation, not a word-for-word transcript of the assistant's replies.

Step 1

You: Shared prompt.pdf (agent brief) and CS_Board_Syllabus.pdf. “The two files contain everything needed for the task. Proceed.”

Done: Read both PDFs. Built carbon_logger.py and database_setup.sql to the brief (two related tables, JOIN, aggregates, 13-option menu, exports, error handling). Installed MariaDB and the connector in the cloud environment and tested every menu option with scripted input, including bad inputs. Committed and pushed to branch claude/wizardly-hamilton-6q6g0h.

Step 2

You: use float, set password to root

Done: Changed the numeric columns from DECIMAL to FLOAT in both files. The password was already root. Re-tested and pushed.

Step 3

You: Review code once again, check for hidden bugs and clean any unwanted lines. Before edits please confirm.

Done: Reviewed line by line and tested suspected cases. Reported 4 bugs (a long note lost the record; nan/inf accepted; export crashed on ₹/Hindi text on Windows; Ctrl+C showed a traceback) and 4 cleanups (duplicate grand-total loop, an extra query, validate_date called twice, .title() damaging names). Made no edits, as asked.

Step 4

You: which all are critical?

Done: Ranked them: #3 (export crash) critical; #1 moderate; #2 low; #4 cosmetic; 5–8 not bugs.

Step 5

You: 1,3,4,5 to 8 apply (make sure edits are within CBSE scope)

Done: Applied them with syllabus-level code only: [:100] slicing and length checks, encoding="utf-8", except KeyboardInterrupt, a get_grand_total() helper, and the small tidy-ups. Tested each and pushed. #2 was left as chosen.

Step 6

You: Is the code ship and presentation ready?

Done: Yes, with a pre-demo checklist: check DB_PASSWORD, install the connector, load the SQL script once, and do a practice run on the actual PC. Noted that CBSE also expects a project report.

Step 7

You: Asked to add the suggested viva sheet to the repo.

Done: Wrote VIVA_SHEET.md (46 questions at the time) and a demo order, then pushed.

Step 8

You: Shared simplify.skill (ASD-STE100 Simplified Technical English): “update the readme with the entire project explained especially the tricky bits.”

Done: Wrote README.md in STE style: glossary, setup with Warning/Caution notes, menu table, database design, program flow, 18 'tricky parts' sections, SQL map, troubleshooting and limitations.

Step 9

> **You:** merge with main

Done: There was no main branch yet, so main was created from the work branch. Noted that GitHub's default branch must be switched to main in Settings.

Step 10

> **You:** Asked for the full CBSE report (introduction, table structures, code, screenshots, bibliography) as one document in the repo.

Done: Captured real program output, rendered terminal-style screenshots and built Project_Report.docx: cover, certificate, acknowledgement, index, 12 sections, full code, figures, tests and bibliography. Fixed a broken LibreOffice install to check the pages. Pushed to both branches.

Step 11

> **You:** also add a PDF version of the report

Done: Added Project_Report.pdf (fonts embedded).

Step 12

> **You:** nvm, remove the pdf keep docx

Done: Removed the PDF and its README line.

Step 13

> **You:** Screenshot of GitHub's empty “Add a README” box: “the readme file should come here instead”

Done: Moved README.md to the repository root, made the file list clickable, and said which folder to open in the setup steps.

Step 14

> **You:** merge with main

Done: Checked: main and the work branch were already identical.

Step 15

> **You:** what is gitignore file for

Done: Explained that it lists files Git should not track: the generated CSV and text summary and Python's __pycache__ folder.

Step 16

> **You:** Shared another version (carbon_logger.py, emission_factors.csv, requirements.txt, setup.sql): “compare yours with this. No edits only comparison. Which is better suited”

Done: Compared them against the brief and the syllabus. Ours is better suited: the other has one table (no foreign key, JOIN or GROUP BY), no search, update or text export, fixed dates, matplotlib, and advanced Python (type hints, pathlib, defaultdict, lambda). Its best idea was an undo feature using a stack (a syllabus topic).

Step 17

> **You:** i think stack option is good, use it

Done: Added menu option 14, Undo Last Added Record: undo_stack list, push() and pop() functions, LIFO undo with confirmation, plus the empty-stack and already-deleted cases. Updated the README, viva sheet (new Stack section) and report (new screenshot, functions and tests). Merged a README edit you had made on GitHub instead of overwriting it.

Step 18

You: is the code still within CBSE scope. After answering this run a last test run and report.

Done: Scope: only csv, datetime and mysql.connector; no classes, lambdas, comprehensions, f-strings or type hints. Wrote an automated end-to-end test: 45/45 checks passed (the 4 early failures were mistakes in the test script). pyflakes clean. Synced your second GitHub README edit.

Step 19

You: wait, what is lastrowid? you said its out of syllabus

Done: Clarified: cursor.lastrowid gives the AUTO_INCREMENT ID of the row just inserted. It isn't on the syllabus list, but it is the same kind of attribute as rowcount. Offered SELECT MAX(record_id) as a syllabus-only alternative.

Step 20

You: keep lastrowid no changes, does the read me and viva sheet contain all the details?

Done: Audited both files against the code and found 8 gaps (connector calls, x if c else y, NULL aggregates, fetchall()[0], MONTH_NAMES[month-1], lastrowid and the syllabus, the nan limitation).

Step 21

You: add all

Done: Added README sections 10.20 and 10.21 and five viva questions, updated the limitations, and renumbered the viva sheet to 59 questions. Pushed.

Step 22

You: How does one run the project on own device? Export all our chat and everything into a pdf … (don't push to repo) Include a brief of the project and code to provide background info

Done: This handbook (first version), kept out of the repository at the time.

Step 23

You: merge with main

Done: Checked: main and the work branch were already identical (the handbook was not in the repo).

Step 24

You: check for bugs

Done: Re-ran all 45 checks under MySQL 8's stricter default settings (all passed) and probed new edge cases. Found: (A) ₹/Hindi notes rejected on servers that default to latin1, such as MySQL 5.7; (B) Ctrl+D / Ctrl+Z showed an EOFError traceback; (C) future dates accepted; (D) a year typed as 0026 silently became 2026; (E) quantities like 1e39 rejected by MySQL; (F) the zero-value message said 'Quantity' for emission factors. Made no edits.

Step 25

You: ACDBF update all documentation as well. then merge with main

Done: Fixed A (CREATE DATABASE … CHARACTER SET utf8mb4), B (except (KeyboardInterrupt, EOFError)), C+D (validate_date accepts 2000 up to today) and F (neutral message). Added 10 tests: 55/55 passed under both MariaDB and MySQL 8 modes. Updated the README (new section 10.22), viva sheet (61 questions) and report (39 pages, 25 tests, new screenshots). Merged into main. E was left as before.

Step 26

> **You:** yes, upload in repo asw

Done: Updated this handbook to the current version and added it to the repository.

Step 27

> **You:** merge with main / did u update readme and everything?

Done: Confirmed main and the work branch were identical, and checked every document against the code: all up to date.

Step 28

> **You:** fix the readme indentation and merge with main

Done: Made the closing disclaimer its own paragraph and merged into main.

Step 29

> **You:** Can I close this or are there any other changes. Also check if the project aligns with the syllabus

Done: Found a new README edit on main (glossary replaced) and listed the syllabus coverage: core Unit 1 and Unit 3 topics covered; not covered: binary files, reading text/CSV files, default parameters, ALTER/DROP/SHOW/DESCRIBE, HAVING, IS NULL, natural join. Noted that the syllabus expects groups of 2–3.

Step 30

> **You:** Keep the edit and merge it. / leave it as it is

Done: Merged the README edit. Left the syllabus gaps as they are.

Step 31

> **You:** Bug check, suggested fixes must be in syllabus scope

Done: Found (1) nan, inf and 1e39 accepted as quantities/factors, then rejected by MySQL with confusing messages; (2) a database error inside menu 12 dropped back to the main menu. Flagged that the earlier CHARACTER SET utf8mb4 clause is outside the syllabus list.

Step 32

> **You:** Apply all

Done: Added MAX_VALUE = 100000 with one comparison (also rejects nan and inf), try/except in both sub-menus, and removed CHARACTER SET (MySQL 8 recommended instead). 59/59 checks passed in both modes. Updated the README (10.22 rewritten), viva sheet (62 questions), report (26 tests) and this handbook; merged into main.

Your own changes on GitHub

- Edited the README introduction to “It is my CBSE Class XII Computer Science project…” and removed the STE note.
- Removed the README limitation line about CSV number formatting (10.0, 1.7).
- Added a disclaimer at the end of the README: “Please note, this is just a small project, data produced may be inaccurate.”
- Replaced the README glossary (section 1) with “Positive Affirmation – Have a good day!”.
- All four edits were merged into the work branch and kept.

5. Key decisions, test results and known limits

Design decisions

Decision	Reason
Two tables + foreign key + INNER JOIN	Required by the brief; shows relational concepts from the syllabus.
Emission factors stored in MySQL	Changeable from menu 12 without editing code.
Old records keep their CO2 when a factor changes	A record shows the estimate made at the time.
FLOAT columns	Your choice; values display cleanly with {:.2f}.
%s placeholders everywhere	Prevents SQL injection and handles quotes in text.
cursor.lastrowid kept	Standard and short; same kind as rowcount (documented in README 10.20 and the viva sheet).
Undo with a list stack (push/pop)	Covers the syllabus Stack topic in a real feature.
Notes cut to 100 characters	The column is VARCHAR(100); long notes no longer lose the record.
UTF-8 for exports	₹ and Hindi text no longer crash the export on Windows.
No CHARACTER SET clause; MySQL 8 recommended	CHARACTER SET is outside the syllabus; MySQL 8 already defaults to utf8mb4 (₹ and Hindi).
Quantities and factors limited to MAX_VALUE = 100000	One syllabus-level comparison rejects nan, inf and numbers MySQL cannot store.
try/except inside both sub-menus	A database error keeps you in the sub-menu instead of dropping to the main menu.
Dates only from 2000 up to today	Rejects future dates and typing mistakes such as 0026-09-15.
Ctrl+D / Ctrl+Z handled like Ctrl+C	Ends the program cleanly and still closes the connection.

Final test run (59 of 59 checks passed, under both MariaDB and MySQL 8 default settings)

Area	Checked
Startup	Program creates the DB and categories itself; the SQL script loads 6 categories + 5 records
Empty database	View, summaries 6–9, exports and undo show proper messages
Add	Bad category, impossible date, text/negative/zero quantity, blank date = today, cancel, 150-char note, ₹/Hindi note
Dates	Future, pre-2000 and 0026 dates rejected; today and 2000-01-01 accepted; update keeps the old date for a future one
View / Search	JOIN output; by ID, category (LIKE), date, date range with swap (BETWEEN); invalid sub-menu choice
Update / Delete	Missing ID, invalid date kept, CO2 recalculated, bad quantities rejected; delete cancel/confirm
Summaries	Overall, category share %, monthly (invalid month, empty month), highest + suggestions
Exports	CSV header + all rows with Unicode intact; text summary complete
Factors	View, update, add, duplicate, too long, empty; new factor applies to new records only
Undo (stack)	LIFO order, cancel, already-deleted record, empty stack
Number limit	nan, inf, -inf, 1e39 and 100001 rejected; 100000 accepted; update keeps the old quantity; factor limit in menu 12
Sub-menu errors	A real database error inside menu 12 is shown and the program stays in menu 12
Other	Help, invalid main-menu input, wrong password, Ctrl+C, Ctrl+D/Ctrl+Z (end of input), real 'python carbon_logger.py' run; pyflakes clean

Known limits

- Emission factors are illustrative, not official data.
- Single user, no login, terminal only.
- Undo works only for records added since the program started (the stack is kept in memory).
- Quantities and emission factors are limited to 100000.
- On servers older than MySQL 8 (latin1), notes can contain English letters only.
- Size is about 695 lines (645 Python + 50 SQL), slightly over the brief's “approximately 500–650”.

Before you submit

1. Fill in the [bracketed] placeholders in Project_Report.docx (school, name, section, roll number, teacher, principal).
2. Check DB_PASSWORD on the presentation PC.
3. Optional: make main the default branch in GitHub → Settings → General.

6. Source code

6.1 database_setup.sql

58 lines

```
-- ============================================================
-- Carbon Data Logger - Database Setup Script
-- Creates the database, both tables and the default categories.
-- Run ONCE in the MySQL client:   SOURCE database_setup.sql;
-- (carbon_logger.py also creates these automatically if missing.)
-- ============================================================

CREATE DATABASE IF NOT EXISTS carbon_logger;
USE carbon_logger;

-- Table 1: activity categories with their unit and emission factor
CREATE TABLE IF NOT EXISTS categories (
    category_id     INT AUTO_INCREMENT PRIMARY KEY,
    category_name   VARCHAR(30) NOT NULL UNIQUE,
    unit            VARCHAR(10) NOT NULL,
    emission_factor FLOAT NOT NULL
);

-- Table 2: one row for every activity recorded by the user
CREATE TABLE IF NOT EXISTS emission_records (
    record_id    INT AUTO_INCREMENT PRIMARY KEY,
    record_date  DATE NOT NULL,
    category_id  INT NOT NULL,
    quantity     FLOAT NOT NULL,
    co2_emission FLOAT NOT NULL,
    note         VARCHAR(100),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);

-- Default categories.
-- NOTE: emission factors (kg CO2 per unit) are ILLUSTRATIVE project
-- values for learning purposes only. They can be changed from menu 12.
INSERT INTO categories (category_name, unit, emission_factor) VALUES
    ('Electricity',  'kWh', 0.820),
    ('Car Travel',   'km',  0.170),
    ('Bus Travel',   'km',  0.100),
    ('Train Travel', 'km',  0.040),
    ('LPG',          'kg',  2.980),
    ('Flight',       'km',  0.150);

-- Some sample records for testing (optional)
INSERT INTO emission_records (record_date, category_id, quantity, co2_emission, note) VALUES
    ('2026-08-02', 1, 12.50, 10.25, 'Daily home use'),
    ('2026-08-03', 2, 20.00,  3.40, 'Trip to market'),
    ('2026-08-05', 3, 15.00,  1.50, 'School bus'),
    ('2026-09-01', 5,  2.00,  5.96, 'Cooking gas'),
```

```
    ('2026-09-10', 6, 800.00, 120.00, 'Flight to Mumbai');

-- Useful queries for checking the data
SELECT * FROM categories;

SELECT r.record_id, r.record_date, c.category_name, r.quantity, c.unit, r.co2_emission
FROM emission_records r INNER JOIN categories c ON r.category_id = c.category_id
ORDER BY r.record_date;

SELECT c.category_name, COUNT(*), SUM(r.co2_emission)
FROM emission_records r INNER JOIN categories c ON r.category_id = c.category_id
GROUP BY c.category_name;
```

6.2 carbon_logger.py

774 lines

```
# ============================================================
# CARBON DATA LOGGER - Personal Carbon Footprint Tracking System
# CBSE Class XII Computer Science Project (Python + MySQL)
#
# Records everyday activities (electricity, travel, LPG, flights),
# estimates their CO2 emissions and stores them in MySQL.
# Emission factors are ILLUSTRATIVE project values, not official data.
# ============================================================

import csv
import datetime
import mysql.connector

# MySQL connection settings - change the password to match your system
DB_HOST = "localhost"
DB_USER = "root"
DB_PASSWORD = "root"
DB_NAME = "carbon_logger"

# Largest quantity or emission factor that the program accepts
MAX_VALUE = 100000

# Output file names
CSV_FILE = "carbon_records.csv"
SUMMARY_FILE = "carbon_summary.txt"

# Default categories: (name, unit, illustrative kg CO2 per unit)
DEFAULT_CATEGORIES = [
    ("Electricity", "kWh", 0.82),
    ("Car Travel", "km", 0.17),
    ("Bus Travel", "km", 0.10),
    ("Train Travel", "km", 0.04),
    ("LPG", "kg", 2.98),
    ("Flight", "km", 0.15),
]

# Simple rule-based suggestions for each category
SUGGESTIONS = {
    "Electricity": "Switch off unused lights/fans and prefer LED bulbs and efficient appliances.",
    "Car Travel": "Consider public transport, car-pooling or cycling for suitable journeys.",
    "Bus Travel": "Bus travel is already a good choice; walk or cycle for very short trips.",
    "Train Travel": "Train travel is a low-emission option; keep using it where possible.",
    "LPG": "Use lids while cooking and pressure cookers to save cooking gas.",
    "Flight": "Flights add a lot of CO2; prefer trains for shorter distances when possible.",
}

MONTH_NAMES = ("January", "February", "March", "April", "May", "June", "July",
               "August", "September", "October", "November", "December")

# Common SELECT used to show records with category name and unit (JOIN)
RECORD_QUERY = ("SELECT r.record_id, r.record_date, c.category_name, r.quantity, "
                "c.unit, r.co2_emission, r.note "
                "FROM emission_records r INNER JOIN categories c "
                "ON r.category_id = c.category_id ")

connection = None
cursor = None

# Stack of record IDs added in this session, used by "Undo Last Added Record"
undo_stack = []


# ---------------- Database setup ----------------

# Connect to the MySQL server and select (or create) the database
def connect_database():
    global connection, cursor
    connection = mysql.connector.connect(host=DB_HOST, user=DB_USER, password=DB_PASSWORD)
    cursor = connection.cursor()
    cursor.execute("CREATE DATABASE IF NOT EXISTS " + DB_NAME)
    cursor.execute("USE " + DB_NAME)


# Create both tables if they do not exist yet
def create_tables():
    cursor.execute("""CREATE TABLE IF NOT EXISTS categories (
```

```
                        category_id INT AUTO_INCREMENT PRIMARY KEY,
                        category_name VARCHAR(30) NOT NULL UNIQUE,
                        unit VARCHAR(10) NOT NULL,
                        emission_factor FLOAT NOT NULL)""")
    cursor.execute("""CREATE TABLE IF NOT EXISTS emission_records (
                        record_id INT AUTO_INCREMENT PRIMARY KEY,
                        record_date DATE NOT NULL,
                        category_id INT NOT NULL,
                        quantity FLOAT NOT NULL,
                        co2_emission FLOAT NOT NULL,
                        note VARCHAR(100),
                        FOREIGN KEY (category_id) REFERENCES categories(category_id))""")
    connection.commit()


# Insert the default categories only when the table is empty
def insert_default_categories():
    cursor.execute("SELECT COUNT(*) FROM categories")
    count = cursor.fetchone()[0]
    if count == 0:
        query = "INSERT INTO categories (category_name, unit, emission_factor) VALUES (%s, %s, %s)"
        for category in DEFAULT_CATEGORIES:
            cursor.execute(query, category)
        connection.commit()
        print("Default categories added.")


# ---------------- Input helper functions ----------------

# Wait for the user before showing the menu again
def pause_program():
    input("\nPress Enter to continue...")


def print_heading(title):
    print("\n" + "=" * 60)
    print(title.center(60))
    print("=" * 60)


# Keep asking until the user types a whole number
def get_integer(prompt):
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Invalid input! Please enter a whole number.")


# Keep asking until the user types a number above zero and not above MAX_VALUE
def get_positive_number(prompt):
    while True:
        try:
            value = float(input(prompt))
            # nan fails both comparisons and inf fails the second, so both are rejected
            if value > 0 and value <= MAX_VALUE:
                return value
            print("The value must be greater than 0 and at most", MAX_VALUE)
        except ValueError:
            print("Invalid input! Please enter a numeric value.")


# Return the date as 'YYYY-MM-DD' if it is valid, otherwise None.
# Only dates from the year 2000 up to today are accepted.
def validate_date(date_text):
    try:
        date_value = datetime.datetime.strptime(date_text, "%Y-%m-%d").date()
    except ValueError:
        return None
    if date_value.year < 2000 or date_value > datetime.date.today():
        return None
    return str(date_value)


# Ask for a date; an empty answer means today's date
def get_date(prompt):
    while True:
        date_text = input(prompt + " (YYYY-MM-DD, Enter for today): ").strip()
        if date_text == "":
            return str(datetime.date.today())
```

```
157         valid_date = validate_date(date_text)
158         if valid_date is None:
159             print("Invalid date! Use YYYY-MM-DD, from the year 2000 up to today.")
160         else:
161             return valid_date
162 
163 
164 def ask_yes_no(prompt):
165     answer = input(prompt + " (Y/N): ").strip().upper()
166     return answer == "Y"
167 
168 
169 # ---------------- Categories ----------------
170 
171 # Display all categories in a table and return them as a list
172 def display_categories():
173     cursor.execute("SELECT category_id, category_name, unit, emission_factor "
174                    "FROM categories ORDER BY category_id")
175     categories = cursor.fetchall()
176     print("\n{:<5}{:<16}{:<8}{:>22}".format("ID", "CATEGORY", "UNIT", "FACTOR (kg CO2/unit)"))
177     print("-" * 51)
178     for category in categories:
179         print("{:<5}{:<16}{:<8}{:>22.3f}".format(category[0], category[1],
180                                                 category[2], category[3]))
181     return categories
182 
183 
184 # Fetch one category (id, name, unit, factor) from MySQL, or None
185 def get_category(category_id):
186     cursor.execute("SELECT category_id, category_name, unit, emission_factor "
187                    "FROM categories WHERE category_id = %s", (category_id,))
188     return cursor.fetchone()
189 
190 
191 # Ask the user to pick a valid category ID
192 def choose_category():
193     display_categories()
194     while True:
195         category_id = get_integer("\nEnter category ID: ")
196         category = get_category(category_id)
197         if category is None:
198             print("No category with that ID. Try again.")
199         else:
200             return category
201 
202 
203 # ---------------- Carbon calculation ----------------
204 
205 # Calculate carbon emission: CO2 = quantity x emission factor
206 def calculate_emission(quantity, emission_factor):
207     return round(float(quantity) * float(emission_factor), 2)
208 
209 
210 # ---------------- Add and view records ----------------
211 
212 def add_record():
213     print_heading("ADD CARBON RECORD")
214     category = choose_category()
215     record_date = get_date("Enter date")
216     quantity = get_positive_number("Enter quantity in " + category[2] + ": ")
217     # Only the first 100 characters are kept because the note column is VARCHAR(100)
218     note = input("Enter a note (optional): ").strip()[:100]
219 
220     # The emission factor comes from the categories table in MySQL
221     co2 = calculate_emission(quantity, category[3])
222     print("\nCalculation:", quantity, category[2], "x", category[3], "kg/" + category[2],
223           "=", co2, "kg CO2")
224 
225     if ask_yes_no("Save this record?"):
226         query = ("INSERT INTO emission_records (record_date, category_id, quantity, "
227                  "co2_emission, note) VALUES (%s, %s, %s, %s, %s)")
228         values = (record_date, category[0], quantity, co2, note)
229         cursor.execute(query, values)
230         connection.commit()
231         print("Record saved successfully with ID", cursor.lastrowid)
232         # Remember the new record so that it can be undone later
233         push(undo_stack, cursor.lastrowid)
234     else:
235         print("Record not saved.")
236 
```

```
237
238 # Print a list of records as a neat table
239 def print_records(records):
240     if len(records) == 0:
241         print("\nNo records found.")
242         return
243     print("\n{:<5}{:<13}{:<15}{:>10}  {:<6}{:>10}  {}".format(
244         "ID", "DATE", "CATEGORY", "QUANTITY", "UNIT", "CO2 (kg)", "NOTE"))
245     print("-" * 80)
246     for record in records:
247         note = record[6] if record[6] else ""
248         print("{:<5}{:<13}{:<15}{:>10.2f}  {:<6}{:>10.2f}  {}".format(
249             record[0], str(record[1]), record[2], record[3], record[4], record[5], note))
250     print("-" * 80)
251     print("Records shown:", len(records))
252
253
254 # Display all stored records
255 def view_all_records():
256     print_heading("ALL CARBON RECORDS")
257     cursor.execute(RECORD_QUERY + "ORDER BY r.record_date, r.record_id")
258     records = cursor.fetchall()
259     print_records(records)
260
261
262 # Fetch one record by its ID, or None if it does not exist
263 def get_record(record_id):
264     cursor.execute(RECORD_QUERY + "WHERE r.record_id = %s", (record_id,))
265     return cursor.fetchone()
266
267
268 # ---------------- Search ----------------
269
270 def search_by_id():
271     record_id = get_integer("Enter record ID: ")
272     record = get_record(record_id)
273     if record is None:
274         print("No record found with ID", record_id)
275     else:
276         print_records([record])
277
278
279 def search_by_category():
280     name = input("Enter category name (or part of it): ").strip()
281     cursor.execute(RECORD_QUERY + "WHERE c.category_name LIKE %s ORDER BY r.record_date",
282                    ("%" + name + "%",))
283     print_records(cursor.fetchall())
284
285
286 def search_by_date():
287     record_date = get_date("Enter date")
288     cursor.execute(RECORD_QUERY + "WHERE r.record_date = %s", (record_date,))
289     print_records(cursor.fetchall())
290
291
292 def search_by_date_range():
293     start_date = get_date("Enter start date")
294     end_date = get_date("Enter end date")
295     if start_date > end_date:
296         print("Start date was after end date, so the dates have been swapped.")
297         start_date, end_date = end_date, start_date
298     cursor.execute(RECORD_QUERY + "WHERE r.record_date BETWEEN %s AND %s "
299                    "ORDER BY r.record_date", (start_date, end_date))
300     print_records(cursor.fetchall())
301
302
303 def search_menu():
304     while True:
305         print_heading("SEARCH RECORDS")
306         print("1. Search by Record ID")
307         print("2. Search by Category")
308         print("3. Search by Date")
309         print("4. Search between two dates")
310         print("5. Return to Main Menu")
311         choice = input("\nEnter your choice: ").strip()
312         try:
313             if choice == "1":
314                 search_by_id()
315             elif choice == "2":
316                 search_by_category()
```

```
            elif choice == "3":
                search_by_date()
            elif choice == "4":
                search_by_date_range()
            elif choice == "5":
                return
            else:
                print("Invalid choice! Please enter 1 to 5.")
        except mysql.connector.Error as error:
            # Stay in this menu after a database error
            print("Database error:", error)
        pause_program()


# ---------------- Update and delete ----------------

def update_record():
    print_heading("UPDATE RECORD")
    record_id = get_integer("Enter record ID to update: ")
    record = get_record(record_id)
    if record is None:
        print("No record found with ID", record_id)
        return
    print("\nCurrent details:")
    print_records([record])

    # The category ID is not part of RECORD_QUERY, so read it separately
    cursor.execute("SELECT category_id FROM emission_records WHERE record_id = %s",
                   (record_id,))
    category_id = cursor.fetchone()[0]
    quantity = record[3]
    new_date = str(record[1])
    new_note = record[6]

    print("\nPress Enter to keep the current value.")
    date_text = input("New date (YYYY-MM-DD): ").strip()
    if date_text != "":
        valid_date = validate_date(date_text)
        if valid_date is None:
            print("Invalid date, keeping the old date.")
        else:
            new_date = valid_date

    if ask_yes_no("Change the category?"):
        category_id = choose_category()[0]

    quantity_text = input("New quantity: ").strip()
    if quantity_text != "":
        try:
            new_quantity = float(quantity_text)
            if new_quantity > 0 and new_quantity <= MAX_VALUE:
                quantity = new_quantity
            else:
                print("Quantity must be greater than 0 and at most", MAX_VALUE, "- keeping the old quantity.")
        except ValueError:
            print("Not a number, keeping the old quantity.")

    note_text = input("New note: ").strip()[:100]
    if note_text != "":
        new_note = note_text

    # Recalculate CO2 using the factor of the (possibly new) category
    category = get_category(category_id)
    new_co2 = calculate_emission(quantity, category[3])

    query = ("UPDATE emission_records SET record_date = %s, category_id = %s, "
             "quantity = %s, co2_emission = %s, note = %s WHERE record_id = %s")
    cursor.execute(query, (new_date, category_id, quantity, new_co2, new_note, record_id))
    connection.commit()
    print("\nRecord updated. Recalculated CO2:", new_co2, "kg")
    print_records([get_record(record_id)])


def delete_record():
    print_heading("DELETE RECORD")
    record_id = get_integer("Enter record ID to delete: ")
    record = get_record(record_id)
    if record is None:
        print("No record found with ID", record_id)
        return
```

```
397      print_records([record])
398      if ask_yes_no("Delete this record?"):
399          cursor.execute("DELETE FROM emission_records WHERE record_id = %s", (record_id,))
400          connection.commit()
401          print(cursor.rowcount, "record deleted.")
402      else:
403          print("Delete cancelled.")
404
405
406  # ---------------- Undo using a stack ----------------
407
408  # Add an item to the top of the stack
409  def push(stack, item):
410      stack.append(item)
411
412
413  # Remove and return the top item of the stack, or None if it is empty
414  def pop(stack):
415      if len(stack) == 0:
416          return None
417      return stack.pop()
418
419
420  # Remove the most recently added record (Last In, First Out)
421  def undo_last_record():
422      print_heading("UNDO LAST ADDED RECORD")
423      record_id = pop(undo_stack)
424      if record_id is None:
425          print("Nothing to undo. No records have been added in this session.")
426          return
427      record = get_record(record_id)
428      if record is None:
429          print("Record", record_id, "was already deleted.")
430          return
431      print_records([record])
432      if ask_yes_no("Remove this record?"):
433          cursor.execute("DELETE FROM emission_records WHERE record_id = %s", (record_id,))
434          connection.commit()
435          print("Record removed.", len(undo_stack), "more record(s) can be undone.")
436      else:
437          # Put the ID back so the user can undo it later
438          push(undo_stack, record_id)
439          print("Undo cancelled.")
440
441
442  # ---------------- Summaries ----------------
443
444  # Return (count, total, average, minimum, maximum) using aggregate functions
445  def get_overall_totals():
446      cursor.execute("SELECT COUNT(*), SUM(co2_emission), AVG(co2_emission), "
447                     "MIN(co2_emission), MAX(co2_emission) FROM emission_records")
448      return cursor.fetchone()
449
450
451  # Return a list of (category, number of records, total CO2), highest first
452  def get_category_totals():
453      cursor.execute("SELECT c.category_name, COUNT(*), SUM(r.co2_emission) "
454                     "FROM emission_records r INNER JOIN categories c "
455                     "ON r.category_id = c.category_id "
456                     "GROUP BY c.category_name ORDER BY SUM(r.co2_emission) DESC")
457      return cursor.fetchall()
458
459
460  # Add up the total CO2 of all categories
461  def get_grand_total(category_totals):
462      grand_total = 0
463      for row in category_totals:
464          grand_total = grand_total + float(row[2])
465      return grand_total
466
467
468  def overall_summary():
469      print_heading("CARBON FOOTPRINT SUMMARY")
470      count, total, average, lowest, highest = get_overall_totals()
471      if count == 0:
472          print("No records available yet.")
473          return
474      print("Total records              :", count)
475      print("Total CO2 emissions        : {:.2f} kg".format(total))
476      print("Average CO2 per record     : {:.2f} kg".format(average))
```

```
    print("Lowest individual emission : {:.2f} kg".format(lowest))
    print("Highest individual emission: {:.2f} kg".format(highest))

    cursor.execute("SELECT MIN(record_date), MAX(record_date) FROM emission_records")
    first_date, last_date = cursor.fetchone()
    print("Records from", first_date, "to", last_date)


def category_summary():
    print_heading("CATEGORY-WISE SUMMARY")
    rows = get_category_totals()
    if len(rows) == 0:
        print("No records available yet.")
        return
    grand_total = get_grand_total(rows)
    print("{:<18}{:>10}{:>16}{:>12}".format("CATEGORY", "RECORDS", "TOTAL CO2", "SHARE"))
    print("-" * 56)
    for row in rows:
        share = float(row[2]) / grand_total * 100 if grand_total > 0 else 0
        print("{:<18}{:>10}{:>16.2f}{:>11.1f}%".format(row[0], row[1], row[2], share))
    print("-" * 56)
    print("{:<18}{:>10}{:>16.2f}".format("TOTAL", "", grand_total))


# Show which months have records, using DISTINCT
def show_available_months():
    cursor.execute("SELECT DISTINCT YEAR(record_date), MONTH(record_date) "
                   "FROM emission_records ORDER BY 1, 2")
    months = cursor.fetchall()
    if len(months) > 0:
        print("Months with records:")
        for year, month in months:
            print("  ", MONTH_NAMES[month - 1], year, "(" + str(month) + "/" + str(year) + ")")
    return len(months)


def monthly_summary():
    print_heading("MONTHLY SUMMARY")
    if show_available_months() == 0:
        print("No records available yet.")
        return
    month = get_integer("\nEnter month number (1-12): ")
    if month < 1 or month > 12:
        print("Invalid month! It must be between 1 and 12.")
        return
    year = get_integer("Enter year (e.g. 2026): ")

    cursor.execute("SELECT COUNT(*), SUM(co2_emission), AVG(co2_emission) FROM emission_records "
                   "WHERE MONTH(record_date) = %s AND YEAR(record_date) = %s", (month, year))
    count, total, average = cursor.fetchone()
    print("\nCARBON SUMMARY -", MONTH_NAMES[month - 1].upper(), year)
    if count == 0:
        print("No records for this month.")
        return
    print("\nTotal records:", count)
    print("Total estimated CO2: {:.2f} kg".format(total))
    print("Average emission: {:.2f} kg".format(average))

    # Category with the largest total in that month
    cursor.execute("SELECT c.category_name, SUM(r.co2_emission) "
                   "FROM emission_records r INNER JOIN categories c "
                   "ON r.category_id = c.category_id "
                   "WHERE MONTH(r.record_date) = %s AND YEAR(r.record_date) = %s "
                   "GROUP BY c.category_name ORDER BY SUM(r.co2_emission) DESC", (month, year))
    top = cursor.fetchall()[0]
    print("Highest category:", top[0], "({:.2f} kg)".format(top[1]))


# Return a list of suggestion strings based on the recorded categories
def generate_suggestions(category_totals):
    suggestions = []
    grand_total = get_grand_total(category_totals)
    for row in category_totals:
        name = row[0]
        share = float(row[2]) / grand_total * 100 if grand_total > 0 else 0
        # Suggest only for categories contributing at least 20% of the total
        if share >= 20:
            if name in SUGGESTIONS:
                advice = SUGGESTIONS[name]
            else:
```

```
                advice = "Try to reduce this activity where it is practical."
            suggestions.append(name + " ({:.1f}% of total): ".format(share) + advice)
    return suggestions


def highest_emission_category():
    print_heading("HIGHEST EMISSION CATEGORY")
    rows = get_category_totals()
    if len(rows) == 0:
        print("No records available yet.")
        return
    # Rows are ordered by total CO2 in descending order, so the first is the highest
    print("Category with the highest emissions:", rows[0][0])
    print("Total CO2 from this category: {:.2f} kg in {} record(s)".format(rows[0][2], rows[0][1]))
    if len(rows) > 1:
        print("Lowest category:", rows[-1][0], "({:.2f} kg)".format(rows[-1][2]))

    print("\nSuggestions:")
    for suggestion in generate_suggestions(rows):
        print("-", suggestion)


# ---------------- Export ----------------

# Write all records to a CSV file using the csv module
def export_to_csv():
    print_heading("EXPORT RECORDS TO CSV")
    cursor.execute(RECORD_QUERY + "ORDER BY r.record_date, r.record_id")
    records = cursor.fetchall()
    if len(records) == 0:
        print("No records to export.")
        return
    try:
        with open(CSV_FILE, "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(["Record ID", "Date", "Category", "Quantity", "Unit",
                             "CO2 (kg)", "Note"])
            writer.writerows(records)
        print(len(records), "records exported to", CSV_FILE)
    except IOError:
        print("Error: could not write to", CSV_FILE, "(is it open in another program?)")


# Write an overall and category-wise summary to a text file
def export_summary_to_text():
    print_heading("EXPORT SUMMARY TO TEXT FILE")
    count, total, average, lowest, highest = get_overall_totals()
    if count == 0:
        print("No records available to summarise.")
        return
    category_rows = get_category_totals()
    file = None
    try:
        file = open(SUMMARY_FILE, "w", encoding="utf-8")
        file.write("CARBON DATA LOGGER\n")
        file.write("PERSONAL FOOTPRINT SUMMARY\n")
        file.write("Generated on: " + str(datetime.date.today()) + "\n\n")
        file.write("Total Records: " + str(count) + "\n")
        file.write("Total Estimated CO2: {:.2f} kg\n".format(total))
        file.write("Average CO2: {:.2f} kg\n".format(average))
        file.write("Lowest Emission: {:.2f} kg\n".format(lowest))
        file.write("Highest Emission: {:.2f} kg\n\n".format(highest))
        file.write("CATEGORY SUMMARY\n")
        file.write("----------------\n")
        for row in category_rows:
            file.write("{}: {:.2f} kg ({} records)\n".format(row[0], row[2], row[1]))
        file.write("\nSUGGESTIONS\n")
        file.write("-----------\n")
        for suggestion in generate_suggestions(category_rows):
            file.write("- " + suggestion + "\n")
        file.write("\nNote: emission factors are illustrative project values.\n")
        print("Summary exported to", SUMMARY_FILE)
    except IOError:
        print("Error: could not write to", SUMMARY_FILE)
    finally:
        # The file is closed whether or not an error occurred
        if file is not None:
            file.close()


```

```
637 # ---------------- Manage emission factors ----------------
638
639 def update_emission_factor():
640     category = choose_category()
641     print("Current factor for", category[1], "is", category[3], "kg CO2 per", category[2])
642     new_factor = get_positive_number("Enter new emission factor: ")
643     cursor.execute("UPDATE categories SET emission_factor = %s WHERE category_id = %s",
644                    (new_factor, category[0]))
645     connection.commit()
646     print("Emission factor updated. It will be used for new or edited records.")
647
648
649 def add_category():
650     name = input("Enter new category name: ").strip()
651     unit = input("Enter unit (e.g. kWh, km, kg): ").strip()
652     if name == "" or unit == "":
653         print("Name and unit cannot be empty.")
654         return
655     if len(name) > 30 or len(unit) > 10:
656         print("Name can have at most 30 characters and unit at most 10.")
657         return
658     factor = get_positive_number("Enter emission factor (kg CO2 per " + unit + "): ")
659     try:
660         cursor.execute("INSERT INTO categories (category_name, unit, emission_factor) "
661                        "VALUES (%s, %s, %s)", (name, unit, factor))
662         connection.commit()
663         print("Category", name, "added.")
664     except mysql.connector.IntegrityError:
665         print("A category with this name already exists.")
666
667
668 def manage_emission_factors():
669     while True:
670         print_heading("MANAGE / VIEW EMISSION FACTORS")
671         print("Note: these factors are illustrative values for this project.")
672         print("1. View Emission Factors")
673         print("2. Update an Emission Factor")
674         print("3. Add a New Category")
675         print("4. Return to Main Menu")
676         choice = input("\nEnter your choice: ").strip()
677         try:
678             if choice == "1":
679                 display_categories()
680             elif choice == "2":
681                 update_emission_factor()
682             elif choice == "3":
683                 add_category()
684             elif choice == "4":
685                 return
686             else:
687                 print("Invalid choice! Please enter 1 to 4.")
688         except mysql.connector.Error as error:
689             # Stay in this menu after a database error
690             print("Database error:", error)
691         pause_program()
692
693
694 # ---------------- Help and main menu ----------------
695
696 def show_help():
697     print_heading("HELP")
698     print("* Add a record for an activity, e.g. 15 kWh of electricity or 20 km by car.")
699     print("* CO2 is estimated as: quantity x emission factor of the category.")
700     print("* Dates are entered as YYYY-MM-DD, from 2000 up to today. Press Enter for today.")
701     print("* Use Search, Update and Delete with the record ID shown in the tables.")
702     print("* Summaries show totals overall, per category and per month.")
703     print("* Exports create", CSV_FILE, "and", SUMMARY_FILE, "in this folder.")
704     print("* Emission factors are illustrative and can be changed from option 12.")
705     print("* Option 14 removes records added in this session, newest first.")
706
707
708 def display_main_menu():
709     print("\n" + "=" * 50)
710     print("CARBON DATA LOGGER".center(50))
711     print("=" * 50)
712     print("1. Add Carbon Record")
713     print("2. View All Records")
714     print("3. Search Records")
715     print("4. Update Record")
716     print("5. Delete Record")
```

```
    print("6. Carbon Footprint Summary")
    print("7. Category-wise Summary")
    print("8. Monthly Summary")
    print("9. Highest Emission Category")
    print("10. Export Records to CSV")
    print("11. Export Summary to Text File")
    print("12. Manage/View Emission Factors")
    print("13. Help")
    print("14. Undo Last Added Record")
    print("0. Exit")


def main():
    try:
        # Connect to the MySQL database and prepare the tables
        connect_database()
        create_tables()
        insert_default_categories()
    except mysql.connector.Error as error:
        print("Could not connect to MySQL:", error)
        print("Check that MySQL is running and the password in DB_PASSWORD is correct.")
        return

    # Each menu number is linked to the function that handles it
    menu_actions = {
        "1": add_record, "2": view_all_records, "3": search_menu,
        "4": update_record, "5": delete_record, "6": overall_summary,
        "7": category_summary, "8": monthly_summary, "9": highest_emission_category,
        "10": export_to_csv, "11": export_summary_to_text,
        "12": manage_emission_factors, "13": show_help, "14": undo_last_record,
    }

    try:
        while True:
            display_main_menu()
            choice = input("\nEnter your choice: ").strip()
            if choice == "0":
                print("Thank you for using Carbon Data Logger. Goodbye!")
                break
            elif choice in menu_actions:
                try:
                    menu_actions[choice]()
                except mysql.connector.Error as error:
                    print("Database error:", error)
                # Sub-menus already pause after each action
                if choice not in ("3", "12"):
                    pause_program()
            else:
                print("Invalid choice! Please enter a number from 0 to 14.")
    except (KeyboardInterrupt, EOFError):
        print("\nProgram stopped by the user.")
    finally:
        # Always close the cursor and connection when the program ends
        cursor.close()
        connection.close()


main()
```